

从容器 cgroups 逃逸检测到通用恶意软件 Syscall 异常检测的论文修改任务计划书

修改对象：原 EsCapturer 论文

2026 年 5 月 29 日

1 修改目标与总体定位

原稿的问题背景聚焦于容器环境中的 cgroups escape attack 检测，核心叙事是：攻击者通过内核 upcall、kernel thread、service process、softirq、container engine 等机制，将资源消耗转移到不受容器 cgroup 约束的系统组件中，从而造成资源逃逸和 DoS 风险。现拟将论文整体改写为面向**通用恶意软件的 syscall 异常检测**，即从“特定容器逃逸攻击检测”扩展为“跨场景、跨恶意软件家族的系统调用行为异常检测”。

修改后的论文应突出以下主线：

1. **问题对象变化：**由 container cgroups escape attacks 改为 generic malware syscall anomaly detection。
2. **数据对象变化：**由容器内攻击相关 syscall/kernel function 日志改为进程级或主机级 syscall 序列、参数、返回值、进程关系、文件/网络/IPC 对象等行为日志。
3. **行为语义变化：**由“行为意图是否导致 cgroups 逃逸”改为“syscall 行为单元是否表现出恶意意图或异常执行模式”。
4. **方法创新强化：**在原有序列视图与图视图融合的基础上，加入**大语言模型辅助的行为要素抽取与交织先验引导的自适应视图权重机制**。
5. **实验体系变化：**由五类 cgroups escape attack 实验改为通用恶意软件检测实验，包括恶意软件家族泛化、未知家族检测、跨工作负载泛化、扰动鲁棒性和系统开销评估。

建议新论文的核心定位为：

本文提出一种面向通用恶意软件 syscall 异常检测的行为语义学习框架。该框架首先利用大语言模型从底层 syscall 日志中抽取行为主体、操作、对象、

资源类型、上下文和潜在意图等语义要素，构造细粒度行为单元；随后分别从序列视图和图结构视图学习行为表示，并设计交织先验引导的自适应视图权重机制，在不同的行为单元关系下动态选择更合适的表示来源；最终通过异常检测或分类模块识别恶意软件行为。

2 全局替换与概念重构任务

2.1 论文题目修改

原题目：

EsCapturer: Detecting Cgroups Escape Attacks For Container Clouds Through Behavior-Intention Learning

建议改为以下方向之一：

1. **MalSysGuard: Generic Malware Syscall Anomaly Detection via LLM-assisted Behavior-Intention Learning**
2. **SysIntent: Interleaving-aware Syscall Anomaly Detection for Malware through Behavior-Intention Learning**
3. **MalCaptor: Detecting Malware Syscall Anomalies through LLM-guided Behavior Unit Modeling**

若希望保留原系统名 EsCapturer，需要在文中解释其含义已从“escape capturer”扩展为“execution behavior capturer”，否则建议更换系统名以避免与 cgroups escape 绑定过深。

2.2 关键词修改

原关键词中的 Cgroups escape detection、container security、cloud native 需要替换为更通用的恶意软件检测关键词。建议关键词如下：

Malware detection; syscall anomaly detection; behavior intention learning; large language model; graph-sequence fusion; interleaving-aware representation learning.

2.3 核心术语替换表

原稿术语	修改后术语	修改说明
cgroups escape attacks	malware syscall anomalies	将问题从特定容器逃逸攻击扩展为通用恶意软件异常行为
container behavior	process/system behavior	不再限定容器场景，可覆盖主机、沙箱、服务器和端点环境
container logs	syscall behavior logs	日志来源改为 syscall trace、audit log、eBPF trace、sandbox trace 等
attack-related container events	malware-related events	syscall 事件选择依据由 cgroups 攻击机理变为恶意行为语义与系统资源交互
behavior intention	syscall behavior intention	保留行为意图概念，但解释为 syscall 层面的操作目标和行为语义
cgroups escape detection	malware anomaly detection	检测任务变为恶意/良性分类、异常检测、未知家族检测等
container resource escape	malicious execution deviation	异常含义从资源逃逸改为偏离正常执行语义或表现恶意目标

3 摘要、引言与动机部分修改计划

3.1 摘要修改任务

摘要需要完全重写，避免继续以 cgroups 和容器隔离作为问题起点。新的摘要建议按以下逻辑组织：

1. 恶意软件通过 syscall 与操作系统资源交互，syscall 行为是刻画恶意执行意图的重要观测面。
2. 现有 syscall 异常检测方法多依赖线性序列或统计特征，难以同时刻画行为顺序、对象依赖、跨进程交织和重叠执行关系。

3. 为此提出新的框架：先利用大语言模型抽取 syscall 行为要素，构造行为单元；再分别学习序列视图和图视图；最后通过交织先验引导的自适应视图权重机制进行融合。
4. 实验在通用恶意软件 syscall 数据集和真实系统负载上验证检测效果、泛化能力、消融贡献和运行开销。

摘要中的结果数值需要等新实验完成后再填写，不能沿用原稿 cgroups escape 的 F1-score。

3.2 引言修改任务

引言需要从“容器安全问题”改为“恶意软件 syscall 行为检测问题”。建议使用如下结构：

1. **背景：**恶意软件在执行过程中必须通过 syscall 完成文件读写、进程创建、权限操作、网络连接、内存映射、IPC 交互等关键行为，因此 syscall trace 是恶意软件检测的重要行为证据。
2. **挑战一：底层 syscall 与高层恶意意图之间存在语义鸿沟。**单个 syscall 往往只描述底层操作，难以直接表达加密勒索、持久化、横向移动、命令控制等高层意图。
3. **挑战二：恶意行为既具有顺序性，又具有结构性。**例如文件枚举、权限探测、进程派生和网络连接存在明确先后顺序；而多进程、多对象、多线程的交互又形成图结构依赖。
4. **挑战三：不同关系下应选择不同视图。**当行为单元之间不存在交织关系时，线性序列更能保留完整执行语义；当行为单元之间存在交织关系时，图结构更能刻画重叠执行、对象共享和依赖传播。
5. **本文方案：**提出 LLM-assisted behavior element extraction 和 interleaving-prior guided adaptive view weighting，使模型同时具备语义增强、序列建模、图结构建模和自适应融合能力。

3.3 贡献点重写任务

建议将贡献点重写为四点：

1. 提出一种面向通用恶意软件 syscall 异常检测的行为意图学习框架，将底层 syscall trace 转换为具有主体、操作、对象、资源和意图语义的行为单元。
2. 设计大语言模型辅助的 syscall 行为要素抽取方法，用于缓解底层系统调用与高层恶意意图之间的语义鸿沟。

3. 设计交织先验引导的自适应视图权重机制，为每一对行为单元动态选择序列表示或图表示，在无交织关系时偏向序列视图，在有交织关系时偏向图视图。
4. 在多类恶意软件、良性系统负载、未知家族、扰动攻击和运行开销等实验设置下验证模型的有效性、可解释性和泛化能力。

4 大语言模型辅助行为要素抽取模块设计

4.1 模块位置

该模块应放在方法部分的开头，作为整个框架的第一步。建议章节命名为：

LLM-assisted Syscall Behavior Element Extraction

中文可写为：

大语言模型辅助的 syscall 行为要素抽取

4.2 输入与输出定义

给定原始 syscall 事件：

$$e_t = (pid, ppid, tid, name, args, ret, ts, proc, file, net, user, cwd), \quad (1)$$

其中 $name$ 表示 syscall 名称, $args$ 表示参数, ret 表示返回值, ts 表示时间戳, $proc$ 、 $file$ 、 net 分别表示进程、文件和网络相关上下文。

大语言模型不直接进行最终恶意判定，而是作为行为语义解析器，将原始 syscall 事件或 syscall 片段转换为结构化行为要素：

$$z_t = \langle s_t, o_t, r_t, a_t, c_t, g_t, y_t \rangle, \quad (2)$$

其中：

- s_t ：行为主体，例如进程、线程、用户、容器或可执行文件；
- o_t ：操作类型，例如 read、write、open、connect、execve、mmap、chmod；
- r_t ：行为对象，例如文件、socket、进程、内存区域、注册表项或 IPC 对象；
- a_t ：资源类型，例如 file、network、process、memory、permission、IPC；
- c_t ：上下文条件，例如调用顺序、返回状态、权限状态、路径类型、目标地址；
- g_t ：潜在行为目标，例如 persistence、data collection、privilege operation、network communication、payload execution；
- y_t ：语义标签或行为模板编号，用于后续行为单元构造。

4.3 行为单元构造

将连续或相关的 syscall 事件聚合为行为单元：

$$u_i = \{z_{t_i}, z_{t_i+1}, \dots, z_{t_i+k}\}, \quad (3)$$

每个行为单元表示一个局部行为意图，例如文件枚举、敏感文件读取、进程派生、权限修改、网络连接、payload 加载等。

行为单元的构造依据包括：

- 时间邻近性：事件是否在较短时间窗口内发生；
- 对象一致性：多个 syscall 是否作用于同一文件、进程、socket 或内存对象；
- 语义连续性：LLM 抽取的行为目标 g_t 是否连续或互补；
- 依赖关系：前一事件的返回值、文件描述符、PID、socket 是否被后一事件使用；
- 进程关系：事件是否属于父子进程、线程组或同一执行链。

4.4 LLM 使用边界

为避免方法被质疑为黑盒检测器，应明确：

1. LLM 只用于**语义要素抽取和行为模板归纳**，不直接输出恶意/良性判定。
2. LLM 输入应经过脱敏、归一化和模板化处理，例如将真实路径替换为 <USER_FILE>、<SYSTEM_FILE>、<TEMP_FILE>。
3. 对 LLM 输出建立可复现的模板库，并通过缓存和人工校验减少随机性。
4. 最终检测模型仍然基于可训练的神经网络和明确的损失函数完成。

5 方法部分总体重构计划

修改后的方法部分建议包含六个子模块：

1. Syscall 行为监控与日志归一化；
2. 大语言模型辅助的行为要素抽取；
3. 行为单元构造与交织关系建模；
4. 序列视图编码器；
5. 图结构视图编码器；
6. 交织先验引导的自适应视图权重融合与异常检测。

其中第 2 和第 6 是相较原稿需要重点强化的新创新点。

6 行为单元与交织关系建模

6.1 行为单元定义

设恶意软件或良性程序执行过程中得到的行为单元集合为：

$$\mathcal{U} = \{u_1, u_2, \dots, u_n\}. \quad (4)$$

每个行为单元 u_i 由一组具有语义相关性的 syscall 事件构成。行为单元可以对应以下类型：

- 文件系统行为：文件创建、读取、覆盖、删除、批量枚举；
- 进程行为：进程创建、命令执行、父子进程派生、进程注入准备；
- 网络行为：DNS 查询、连接建立、数据外传、C2 通信；
- 权限行为：权限修改、敏感路径访问、凭据文件读取；
- 内存行为：内存映射、可执行内存申请、动态库加载；
- IPC 行为：pipe、socketpair、shared memory、signal 等。

6.2 交织关系矩阵

定义交织矩阵：

$$C_{ij} = \begin{cases} 1, & \text{若 } u_i \text{ 与 } u_j \text{ 存在交织关系,} \\ 0, & \text{若 } u_i \text{ 与 } u_j \text{ 不存在交织关系.} \end{cases} \quad (5)$$

交织关系可由以下条件判定：

- 两个行为单元在时间窗口上重叠；
- 两个行为单元共享同一对象，例如文件描述符、socket、PID、路径、IP 地址；
- 一个行为单元的输出对象被另一个行为单元使用；
- 两个行为单元属于父子进程、线程组或同一攻击阶段；
- 二者之间存在图边，例如 process-file、process-socket、process-process、file-permission 等。

6.3 关系特征

对每一对行为单元 (u_i, u_j) 构造关系特征：

$$r_{ij} = [\Delta t_{ij}, overlap_{ij}, strength_{ij}, sameObj_{ij}, sameProc_{ij}, edgeType_{ij}, typePair_{ij}], \quad (6)$$

其中 Δt_{ij} 表示时间距离, $overlap_{ij}$ 表示是否重叠, $strength_{ij}$ 表示交织强度, $sameObj_{ij}$ 表示是否同对象, $sameProc_{ij}$ 表示是否同进程, $edgeType_{ij}$ 表示图边类型, $typePair_{ij}$ 表示行为类型组合。

7 序列视图与图视图表示学习

7.1 序列视图编码

序列视图强调行为单元在执行时间上的前后顺序。对行为单元 u_j , 使用序列编码器得到：

$$h_j^s = \text{SeqEncoder}(u_j), \quad (7)$$

其中 h_j^s 表示序列视图表示, 可由 Transformer、BiLSTM、Temporal CNN 或 syscall n-gram Transformer 获得。

该视图主要刻画：

- syscall 的局部顺序模式；
- 行为阶段之间的先后依赖；
- 恶意执行链中的操作序列；
- 正常程序执行流与异常执行流之间的偏差。

7.2 图视图编码

图视图强调行为单元之间的结构依赖、对象共享和交织关系。对行为单元 u_j , 构造局部或全局行为图后得到：

$$h_j^g = \text{GraphEncoder}(u_j), \quad (8)$$

其中 h_j^g 表示图视图表示, 可由 GCN、GAT、R-GCN、Heterogeneous GNN 或 temporal graph encoder 获得。

图视图主要刻画：

- 进程、文件、网络、内存对象之间的依赖；

- 多线程、多进程和多对象交织执行；
- 恶意软件在不同资源之间的传播路径；
- 线性 syscall 序列难以表达的结构关系。

8 交织先验引导的自适应视图权重机制

8.1 设计动机

原稿中图视图和序列视图的融合主要通过注意力或异构特征融合完成，但没有明确说明何时应更依赖序列表示，何时应更依赖图表示。修改后建议将这一部分设计为核心创新点：

对于任意行为单元对 (u_i, u_j) ，模型学习一个面向关系的视图选择权重。当 u_i 与 u_j 不存在交织关系时，模型应更偏向使用 u_j 的序列视图表示；当二者存在交织关系时，模型应更偏向使用 u_j 的图视图表示。

该机制不采用硬切换，而是采用软权重，使模型既能遵循交织先验，又能根据具体样本自适应调整。

8.2 单权重 sigmoid 形式

可先在方法动机中介绍单权重形式。对任意行为单元对 (u_i, u_j) ，学习一个图视图选择权重：

$$w_{ij} \in [0, 1]. \quad (9)$$

其中：

- w_{ij} 越接近 0，越偏向使用 u_j 的序列视图表示；
- w_{ij} 越接近 1，越偏向使用 u_j 的图视图表示。

基础形式为：

$$w_{ij} = \sigma \left(\text{MLP} \left([h_i^s; h_i^g; h_j^s; h_j^g; C_{ij}; r_{ij}] \right) \right). \quad (10)$$

融合表示为：

$$v_{ij} = (1 - w_{ij})h_j^s + w_{ij}h_j^g. \quad (11)$$

进一步将交织先验写入权重计算：

$$w_{ij} = \sigma(\beta(2C_{ij} - 1) + \text{MLP}(r_{ij})). \quad (12)$$

当 $C_{ij} = 0$ 时, $2C_{ij} - 1 = -1$, 因此 w_{ij} 天然偏小, 模型更偏向序列视图; 当 $C_{ij} = 1$ 时, $2C_{ij} - 1 = 1$, 因此 w_{ij} 天然偏大, 模型更偏向图视图。

8.3 推荐采用的双权重 softmax 形式

为了在论文中解释得更直观, 建议最终采用双权重形式:

$$a_{ij}^s = \text{MLP}_s([h_i^s; h_j^s; r_{ij}]) + \beta(1 - C_{ij}), \quad (13)$$

$$a_{ij}^g = \text{MLP}_g([h_i^g; h_j^g; r_{ij}]) + \beta C_{ij}, \quad (14)$$

然后通过 softmax 得到两个视图权重:

$$[w_{ij}^s, w_{ij}^g] = \text{softmax}([a_{ij}^s, a_{ij}^g]). \quad (15)$$

最终构造 u_j 面向 u_i 的关系感知融合表示:

$$v_{ij} = w_{ij}^s h_j^s + w_{ij}^g h_j^g. \quad (16)$$

该设计中:

- 若 $C_{ij} = 0$, 则 a_{ij}^s 获得先验增强, w_{ij}^s 倾向于大于 w_{ij}^g , 模型优先使用序列表示;
- 若 $C_{ij} = 1$, 则 a_{ij}^g 获得先验增强, w_{ij}^g 倾向于大于 w_{ij}^s , 模型优先使用图表示;
- β 控制交织先验强度, β 越大, 模型越遵循“无交织用序列、有交织用图”的先验; β 越小, 模型越依赖数据自适应学习。

8.4 与 cross-attention 融合

注意力权重和视图选择权重应明确区分。对行为单元 u_i 聚合其他行为单元 u_j 时, 先计算注意力分数:

$$\text{score}_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} + b_{ij}, \quad (17)$$

其中结构偏置 b_{ij} 可定义为:

$$b_{ij} = b_{\text{interleave}}(C_{ij}) + b_{\text{temporal}}(i, j) + b_{\text{object}}(i, j) + b_{\text{edge_type}}(i, j). \quad (18)$$

然后计算单元重要性权重:

$$\alpha_{ij} = \text{softmax}_j(\text{score}_{ij}). \quad (19)$$

最终聚合得到：

$$\tilde{h}_i = \sum_j \alpha_{ij} v_{ij}. \quad (20)$$

其中：

- α_{ij} 表示 u_i 应该关注哪些行为单元 u_j ，属于**行为单元重要性权重**；
 - w_{ij}^s, w_{ij}^g 表示关注 u_j 时应优先使用序列表示还是图表示，属于**视图选择权重**。
- 二者不能混在一起，否则方法解释会不清晰。

8.5 先验约束损失

为保证模型符合设计目标，可加入视图选择偏好损失：

$$\mathcal{L}_{pref} = \text{BCE}(w_{ij}^g, C_{ij}), \quad (21)$$

其含义是：

- 若 $C_{ij} = 0$ ，希望图视图权重 w_{ij}^g 接近 0；
- 若 $C_{ij} = 1$ ，希望图视图权重 w_{ij}^g 接近 1。

总损失建议写为：

$$\mathcal{L} = \mathcal{L}_{det} + \lambda_{pref} \mathcal{L}_{pref} + \lambda_{reg} \mathcal{L}_{reg}, \quad (22)$$

其中 $\mathcal{L}_{det}$ 为检测损失。若采用监督式恶意软件检测，可使用交叉熵损失；若采用单类异常检测，可使用 DSVDD 或 one-class objective。 λ_{pref} 控制交织先验约束强度， $\mathcal{L}_{reg}$ 可用于权重平滑、熵约束或过拟合抑制。

8.6 创新点表述建议

论文中可将该机制表述为：

为了刻画行为单元之间顺序关系与交织关系对表示学习的不同影响，本文设计交织关系引导的自适应视图选择机制。对于任意行为单元对，模型分别计算序列视图权重和图视图权重，并利用交织关系作为先验偏置。当两个行为单元不存在交织关系时，模型倾向于选择序列化表示，以保留完整的前后执行语义；当两个行为单元存在交织关系时，模型倾向于选择图结构表示，以

刻画线性序列难以表达的重叠执行和结构依赖。该机制通过软权重而非硬规则实现视图选择，使模型既能遵循行为先验，又能根据具体样本进行自适应调整。

9 系统架构图和方法图修改计划

9.1 原系统架构图修改

原系统架构图围绕 container behavior monitor、behavior correlation、deep-learning-based detection 展开。新架构图建议改为五层：

1. **Syscall Collection Layer**: 从主机、沙箱或端点收集 syscall trace、参数、返回值、进程关系和对象访问日志；
2. **LLM-assisted Element Extraction Layer**: 抽取主体、操作、对象、资源类型、上下文和潜在意图；
3. **Behavior Unit Construction Layer**: 根据时间邻近、对象共享、依赖传播和语义连续性构造行为单元；
4. **Dual-view Representation Layer**: 并行学习序列视图 h^s 和图视图 h^g ；
5. **Interleaving-prior Adaptive Fusion Layer**: 根据 C_{ij} 和 r_{ij} 学习 w_{ij}^s, w_{ij}^g ，再进行 cross-attention 聚合和异常检测。

9.2 新增机制图

建议新增一张核心机制图，展示以下内容：

1. 两个行为单元 u_i, u_j ；
2. 两条编码路径: $h_j^s = \text{SeqEncoder}(u_j)$, $h_j^g = \text{GraphEncoder}(u_j)$ ；
3. 交织矩阵 C_{ij} 与关系特征 r_{ij} ；
4. softmax 视图选择模块输出 w_{ij}^s, w_{ij}^g ；
5. 融合表示 $v_{ij} = w_{ij}^s h_j^s + w_{ij}^g h_j^g$ ；
6. cross-attention 输出 $\tilde{h}_i$ ；
7. 检测头输出 anomaly score 或 malware probability。

10 实验部分修改计划

10.1 实验问题重设

原 RQ 围绕 cgroups escape detection、消融、开销、eBPF emerging threats 展开。修改后建议设置如下 RQs:

1. **RQ1: 检测有效性。**模型能否在通用恶意软件 syscall trace 上准确区分恶意行为和良性行为?
2. **RQ2: 跨家族泛化。**模型能否检测训练集中未出现的恶意软件家族或攻击阶段?
3. **RQ3: 模块贡献。**LLM 行为要素抽取、序列视图、图视图、交织先验权重和偏好损失分别带来多大贡献?
4. **RQ4: 鲁棒性。**面对 syscall 插入、无关噪声、行为重排、低频执行和对象混淆时, 模型是否仍保持稳定?
5. **RQ5: 效率与部署开销。**日志采集、LLM 要素抽取、行为单元构造、模型推理分别带来多少时间和资源开销?
6. **RQ6: 可解释性。**视图权重 w_{ij}^s, w_{ij}^g 是否与交织关系一致, 能否解释模型为何在某些行为关系中偏向图视图或序列视图?

10.2 数据集修改

实验数据应从原来的 Docker image workload 与 cgroups escape attack 日志改为恶意软件 syscall 行为数据。建议设计三类数据来源:

1. **良性系统调用数据:** 包含 Web 服务、数据库、编译任务、文件操作、网络下载、压缩解压、脚本执行、软件安装、浏览器访问等正常工作负载。
2. **恶意软件数据:** 覆盖勒索软件、挖矿木马、后门、下载器、蠕虫、权限提升工具、数据窃取程序、C2 客户端等类型。
3. **公开或自建沙箱数据:** 若使用公开数据集, 应说明其 syscall 字段、标签粒度、家族类别和采集环境; 若自建沙箱, 应说明样本来源、隔离策略、执行时间、网络模拟和清洗流程。

数据划分建议包括:

- 随机划分: 验证基本检测能力;
- 时间划分: 模拟真实部署中未来样本检测;
- 家族隔离划分: 训练集和测试集恶意软件家族不重合, 用于验证未知家族泛化;

- 主机/环境隔离划分：训练和测试来自不同系统环境，用于验证跨环境泛化；
- 少标注或无标注设置：验证异常检测能力，而不仅是监督分类能力。

10.3 对比方法修改

原稿中的 log-based anomaly detection baseline 需要替换为 syscall malware detection 常见基线。建议包括：

1. **统计特征方法**：syscall frequency、TF-IDF、n-gram + SVM、Random Forest、XGBoost；
2. **传统异常检测方法**：OC-SVM、Isolation Forest、HMM；
3. **序列深度模型**：LSTM、GRU、LSTM-AE、Transformer encoder、DeepLog-style model；
4. **图模型**：GNN-based process-object graph detector、GAT/R-GCN detector；
5. **规则方法**：基于 audit rule、sandbox signature 或安全规则的检测器；
6. **本文变体**：sequence-only、graph-only、fusion without interleaving prior、hard switching、sigmoid weighting、softmax weighting、without LLM elements。

10.4 评价指标修改

建议采用以下指标：

- 分类指标：Accuracy、Precision、Recall、F1-score、AUC、FPR、FNR；
- 异常检测指标：AUC-ROC、AUC-PR、Equal Error Rate、Detection Rate at fixed FPR；
- 泛化指标：unknown-family F1、cross-environment F1、family-wise recall；
- 鲁棒性指标：不同扰动强度下的性能下降比例；
- 效率指标：trace collection overhead、feature extraction latency、LLM extraction cost、inference latency、memory usage。

10.5 消融实验修改

消融实验需要围绕新创新点展开。建议表格如下：

模型变体	验证目的
------	------

Full model	完整模型，包含 LLM 要素抽取、序列视图、图视图、交织先验和偏好损失
w/o LLM elements	验证大语言模型抽取的语义要素是否提升检测性能
w/o graph view	验证图结构视图对交织行为、对象依赖和多进程行为的贡献
w/o sequence view	验证序列视图对 syscall 顺序语义的贡献
w/o interleaving prior	去掉 βC_{ij} 和 $\beta(1 - C_{ij})$ ，验证交织先验是否有效
w/o $\mathcal{L}_{pref}$	去掉视图偏好损失，验证权重监督是否有助于符合设计目标
hard switching	使用硬规则替代软权重，验证自适应软选择的优势
sigmoid weighting	使用单权重 w_{ij} ，与双权重 softmax 形式比较
w/o structural bias b_{ij}	验证 attention 中结构偏置的贡献
random C_{ij}	将交织矩阵随机打乱，验证性能提升是否来自真实交织关系而非额外参数

10.6 参数敏感性实验

需要新增以下参数实验：

1. β 的影响：测试 $\beta \in \{0, 0.1, 0.5, 1, 2, 5\}$ ，观察交织先验强度过弱或过强时的性能变化。
2. λ_{pref} 的影响：测试偏好损失权重对 F1、AUC 和权重可解释性的影响。
3. 行为单元窗口大小：测试不同时间窗口或事件数量窗口对行为单元构造的影响。
4. 交织阈值：测试 overlap、same object、dependency strength 等阈值变化对性能的影响。
5. LLM 模板粒度：比较粗粒度行为标签与细粒度行为要素对检测性能的影响。

10.7 可解释性实验

为了支撑交织先验机制，需要新增可解释性实验：

1. 统计 $C_{ij} = 0$ 与 $C_{ij} = 1$ 时 w_{ij}^g 的分布，验证有交织时图权重更高、无交织时序列权重更高。
2. 展示典型恶意软件样本中的行为单元图，例如“文件枚举 → 敏感文件读取 → 网络外传”。

3. 展示 attention heatmap, 说明模型关注哪些行为单元。
4. 展示错误案例, 分析模型在高噪声、短序列或伪装行为下失败的原因。

10.8 鲁棒性实验

建议设置以下扰动:

- syscall insertion: 插入无关良性 syscall;
- syscall deletion: 删除部分非关键 syscall;
- behavior reordering: 对局部可交换行为进行重排;
- low-frequency execution: 降低恶意行为触发频率;
- object aliasing: 改变文件名、路径名、临时目录或网络地址;
- process fragmentation: 将行为拆分到多个子进程中执行。

目标是证明: 单纯依赖序列的模型容易受重排和插入影响, 单纯依赖图的模型可能忽视顺序语义, 而本文模型可通过交织先验自适应选择视图, 从而更稳健。

11 章节级修改清单

原章节	主要问题	修改任务
Abstract	仍围绕 container/cgroups escape	改为通用恶意软件 syscall 异常检测; 加入 LLM 要素抽取和交织先验融合
Introduction	背景限定在容器隔离与 cgroups 漏洞	改为恶意软件通过 syscall 与系统资源交互; 强调语义鸿沟、顺序/结构双重建模挑战
Motivation	攻击机理是 kernel upcall、kernel thread 等 cgroups escape 机制	改为恶意软件行为案例, 例如持久化、勒索加密、C2 通信、数据外传、进程派生
System Overview	系统监控 container behavior	改为 syscall trace collection、LLM element extraction、behavior unit construction、dual-view detection

Behavior Correlation	依据 cgroups 攻击相关事件做相关性分析	改为基于时间、对象、进程、语义意图和数据依赖构造行为单元与交织矩阵
Deep-learning Model	原融合机制创新性不足	增加交织先验引导的自适应视图权重机制, 明确 α_{ij} 与 w_{ij} 的区别
Implementation	绑定容器 PID namespace 和 cgroups 场景	改为主机/沙箱/端点 syscall 采集, 可保留 eBPF/auditd/ptrace 等实现路线
Evaluation	数据和攻击类型均为 cgroups escape	改为恶意软件检测数据集、未知家族、鲁棒性、消融、开销和可解释性实验
Discussion	讨论容器威胁模型和 eBPF 逃逸	改为讨论 LLM 可靠性、恶意软件演化、对抗扰动、跨环境部署和隐私问题
Related Work	重点是 container security	改为 malware behavior detection、syscall anomaly detection、LLM for security、graph/sequence representation learning
Conclusion	总结 cgroups escape 检测	总结通用恶意软件 syscall 异常检测和交织先验自适应融合贡献

12 表格与图的修改清单

图/表	原内容	修改后内容
Figure 1	EsCapturer 容器检测架构	通用恶意软件 syscall 异常检测架构, 包含 LLM 要素抽取层
Figure 2	行为相关性或检测框架	双视图编码 + 交织先验自适应权重 + cross-attention 聚合
新增 Figure	无	视图权重机制示意图: C_{ij} 、 r_{ij} 、 w_{ij}^s 、 w_{ij}^g 、 v_{ij}

Table 1	cgroups 相关 kernel events	syscall 行为要素类型和资源类型表
Table 2	cgroups attack interval	恶意软件类别、行为阶段、样本数量、syscall 数量统计
Table 3	Docker images benchmark	良性工作负载和对应采集场景
主结果表	五类 cgroups attack 检测结果	多类恶意软件/未知家族/跨环境检测结果
消融表	原模型模块消融	LLM、序列视图、图视图、交织先验、 $\mathcal{L}_{pref}$ 、hard switch 消融
新增表	无	β 与 λ_{pref} 参数敏感性结果
新增图	无	C_{ij} 与图视图权重 w_{ij}^g 的分布图

13 写作优先级与任务分解

13.1 第一优先级：问题背景整体改写

1. 删除或弱化 cgroups、container cloud、resource escape、kernel upcall 等特定背景。
2. 将开头改为恶意软件 syscall 行为检测背景。
3. 重新定义 threat model：攻击者控制恶意程序或恶意进程，其行为通过 syscall trace 被监控；攻击目标是逃避检测、窃取数据、持久化、加密文件或建立 C2 通信。
4. 重新定义 anomaly：异常不再是资源逃逸，而是 syscall 行为意图、执行顺序和结构依赖偏离良性程序模式。

13.2 第二优先级：方法创新点重写

1. 增加 LLM-assisted behavior element extraction。
2. 将 behavior intention learning 改写为 syscall behavior intention learning。
3. 重新定义行为单元 u_i 和交织矩阵 C_{ij} 。
4. 将原 heterogeneous feature fusion 替换或升级为 interleaving-prior guided adaptive view weighting。
5. 在方法中完整写出 a_{ij}^s 、 a_{ij}^g 、 w_{ij}^s 、 w_{ij}^g 、 v_{ij} 、 α_{ij} 和 $\mathcal{L}_{pref}$ 。

13.3 第三优先级：实验重构

1. 重新设计数据集、恶意软件类别、良性工作负载和数据划分方式。
2. 重新选择 baseline，避免继续使用只适用于原场景的 cgroups/Falco 对比。
3. 新增 LLM 要素抽取、交织先验、自适应视图权重的消融实验。
4. 新增未知家族、扰动鲁棒性和可解释性实验。
5. 原 cgroups escape 结果可删除，或作为附录中的“特定场景扩展实验”，不能作为主实验。

13.4 第四优先级：讨论与相关工作重写

1. Discussion 中重点讨论 LLM 抽取误差、跨系统泛化、对抗扰动、隐私脱敏和部署成本。
2. Related Work 改为三类：syscall-based malware detection、behavior graph learning、LLM for security log understanding。
3. Conclusion 中强调通用恶意软件 syscall 异常检测，而不是容器 cgroups escape。

14 建议的新论文大纲

1. Introduction

- Malware syscall anomaly detection background
- Semantic gap between syscall and malicious intention
- Sequence-structure duality of malware behavior
- LLM-assisted element extraction and interleaving-aware fusion
- Contributions

2. Motivation

- Example of benign and malicious syscall traces
- Why sequence-only modeling is insufficient
- Why graph-only modeling is insufficient
- Why interleaving relationship should guide view selection

3. System Overview

- Syscall collection
- LLM-assisted element extraction
- Behavior unit construction
- Dual-view representation learning
- Interleaving-prior adaptive fusion
- Detection module

4. Methodology

- Problem formulation
- LLM-assisted behavior element extraction
- Behavior unit and interleaving matrix construction
- Sequence encoder
- Graph encoder
- Adaptive view weighting with interleaving prior
- Detection objective

5. Implementation

- Syscall tracing implementation
- Log normalization and privacy-preserving abstraction
- LLM prompt/template construction
- Model training and inference

6. Evaluation

- Experimental setup and datasets
- Baselines and metrics
- Overall detection performance
- Unknown-family generalization
- Ablation study
- Robustness study
- Efficiency and overhead
- Interpretability analysis

7. Discussion

- LLM reliability and reproducibility
- Limitations under heavily obfuscated malware
- Deployment scenarios
- Privacy and data desensitization

8. Related Work

- Syscall-based malware detection
- Behavior anomaly detection
- Graph and sequence representation learning
- LLM-assisted security analysis

9. Conclusion

15 可直接放入论文的方法段落草稿

下面给出一段可直接改写进方法部分的中文草稿：

给定一段程序执行过程中产生的 syscall 日志，本文首先利用大语言模型从底层事件中抽取行为主体、操作、对象、资源类型、上下文条件和潜在行为目标等语义要素，并根据时间邻近性、对象一致性和依赖关系构造行为单元。对于每个行为单元，模型分别学习序列视图表示和图视图表示，其中序列视图用于刻画 syscall 的前后执行语义，图视图用于刻画进程、文件、网络和内存对象之间的结构依赖。进一步地，考虑到不同的行为单元关系对视图选择具有不同需求，本文设计交织先验引导的自适应视图权重机制。对于任意行为单元对，若二者不存在交织关系，模型倾向于选择序列视图，以保留完整的线性执行语义；若二者存在交织关系，模型倾向于选择图视图，以刻画重叠执行和对象依赖。该机制通过软权重而非硬规则进行融合，使模型既能利用行为先验，又能根据具体样本自适应调整视图选择。

16 预计交付物

1. 一版新的 Introduction、Motivation 和 Contributions；
2. 一版新的 System Overview 图和文字说明；
3. 一节完整的 LLM-assisted Syscall Behavior Element Extraction；

4. 一节完整的 Interleaving-prior Guided Adaptive View Weighting;
5. 一版重构后的 Evaluation, 包括数据集、baseline、RQ、消融和鲁棒性实验;
6. 一版新的 Related Work 和 Discussion;
7. 全文术语统一表, 确保不再残留 cgroups escape、container-specific attack 等旧背景表述。

17 检查清单

最终改稿前应逐项检查:

- 标题、摘要、关键词是否已经完全转向 malware syscall anomaly detection;
- 引言中是否明确提出 syscall 语义鸿沟、顺序/结构双重建模、交织关系视图选择三个核心挑战;
- 是否在方法开头加入 LLM 抽取行为要素;
- 是否定义行为单元 u_i 、序列表示 h_i^s 、图表示 h_i^g 、交织矩阵 C_{ij} 和关系特征 r_{ij} ;
- 是否明确区分 α_{ij} 与 w_{ij}^s, w_{ij}^g ;
- 是否加入 $\mathcal{L}_{pref}$ 并说明其不是硬规则;
- 实验是否从 cgroups escape 改为通用恶意软件 syscall 检测;
- 消融实验是否覆盖 LLM、交织先验、视图权重、序列/图视图和损失项;
- 讨论部分是否补充 LLM 可靠性、隐私脱敏、跨环境泛化和对抗扰动;
- 全文是否删除或转移原有 cgroups/container 过强绑定的论述。